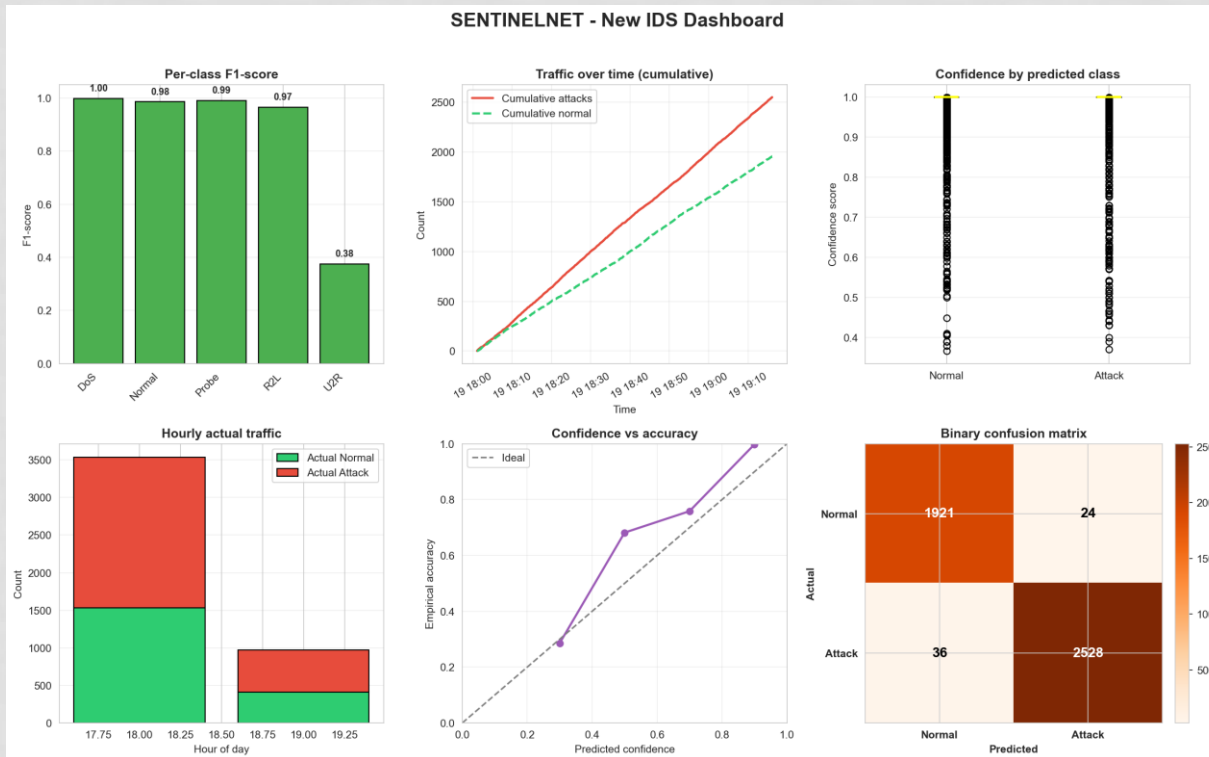


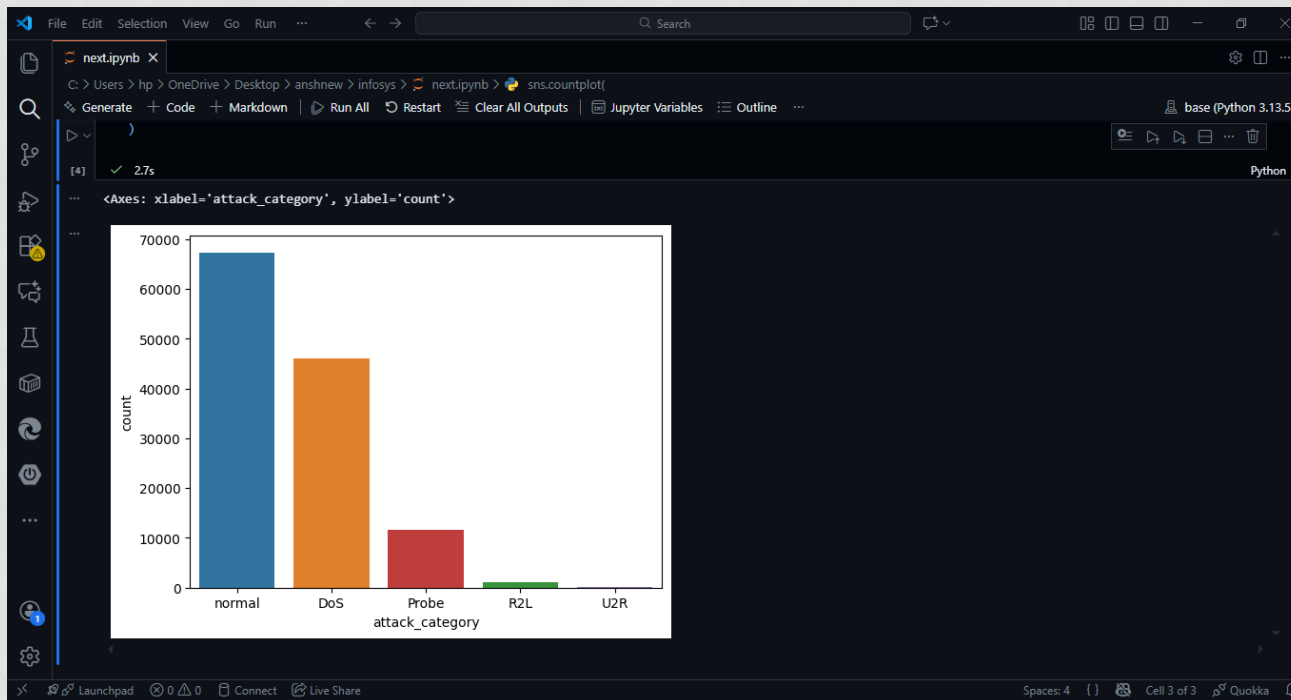
AI-POWERED NETWORK INTRUSION DETECTION SYSTEM



- By : Anshul Bhilare

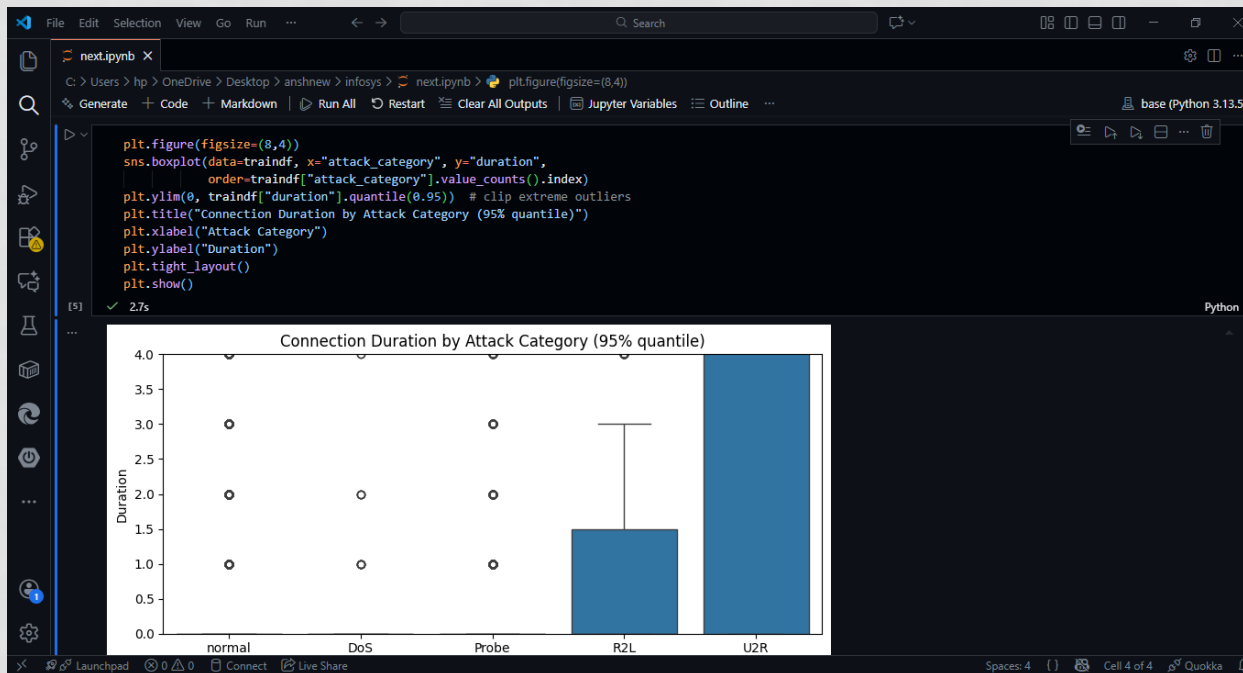
PROJECT OVERVIEW

- SentinelNet is an AI-powered NIDS that detects malicious network traffic in real time using machine learning.



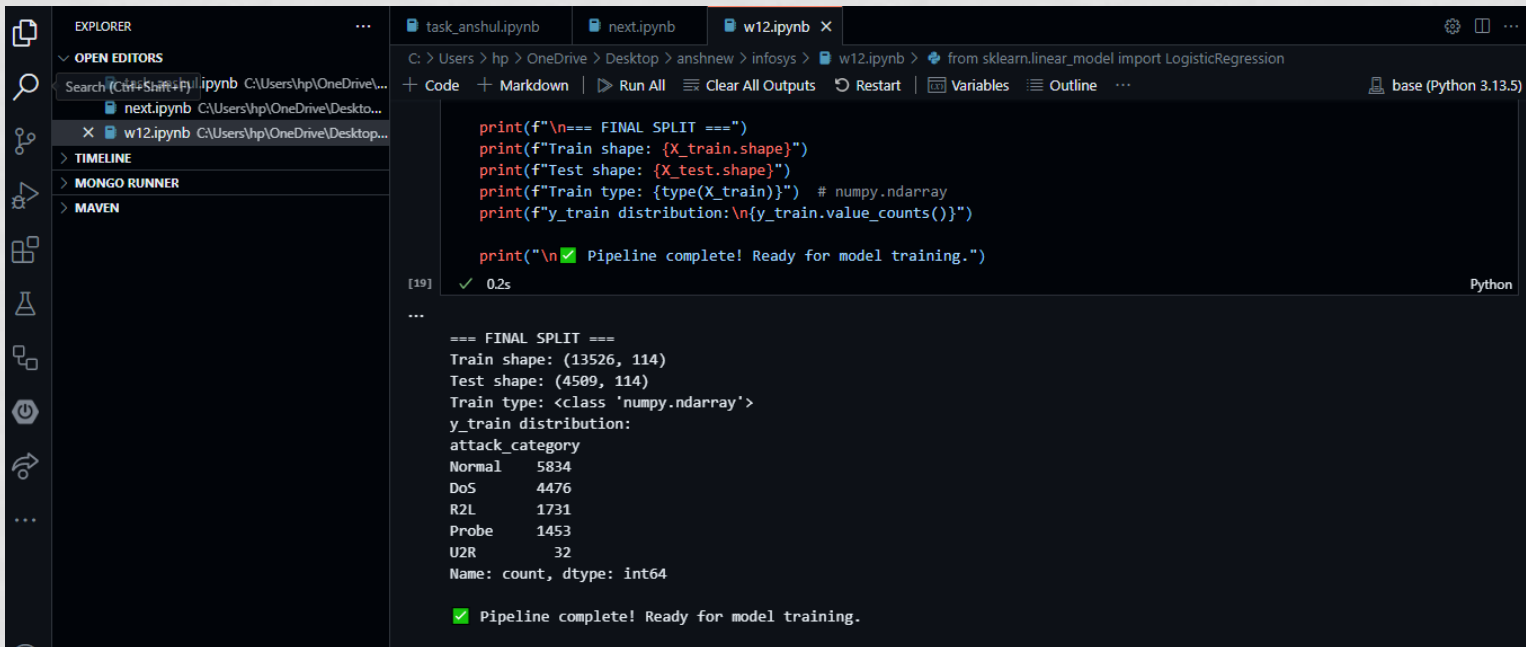
PROBLEM STATEMENT

- Traditional security systems fail to detect modern cyber-attacks.
- Need an intelligent, adaptive solution.



PROJECT OBJECTIVES

- Detect malicious traffic
- Use ML models
- Generate alerts
- Improve network security



The screenshot displays a Jupyter Notebook environment with a dark theme. The left sidebar shows the Explorer and Open Editors panels. The main area contains a Python script for data analysis and model training. The script includes imports for LogisticRegression and LogisticRegressionCV, and uses print statements to display the shape and type of the training and testing data. The output shows the data shapes and types, and a summary of the training data distribution.

```
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LogisticRegressionCV

print("\n=== FINAL SPLIT ===")
print(f"Train shape: {X_train.shape}")
print(f"Test shape: {X_test.shape}")
print(f"Train type: {type(X_train)}") # numpy.ndarray
print(f"y_train distribution:\n{y_train.value_counts()}")

print("\n✅ Pipeline complete! Ready for model training.")
```

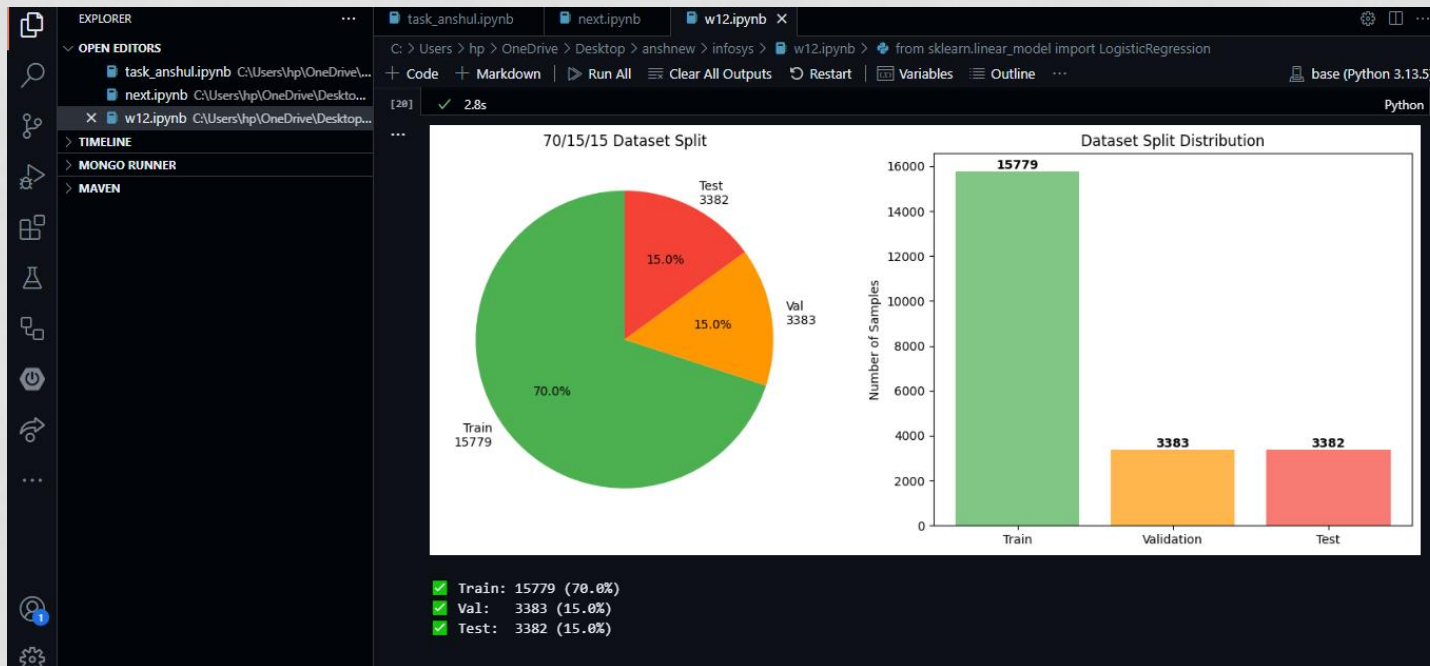
[19] ✓ 0.2s Python

```
...
=== FINAL SPLIT ===
Train shape: (13526, 114)
Test shape: (4509, 114)
Train type: <class 'numpy.ndarray'>
y_train distribution:
attack_category
Normal    5834
DoS       4476
R2L       1731
Probe     1453
U2R        32
Name: count, dtype: int64

✅ Pipeline complete! Ready for model training.
```

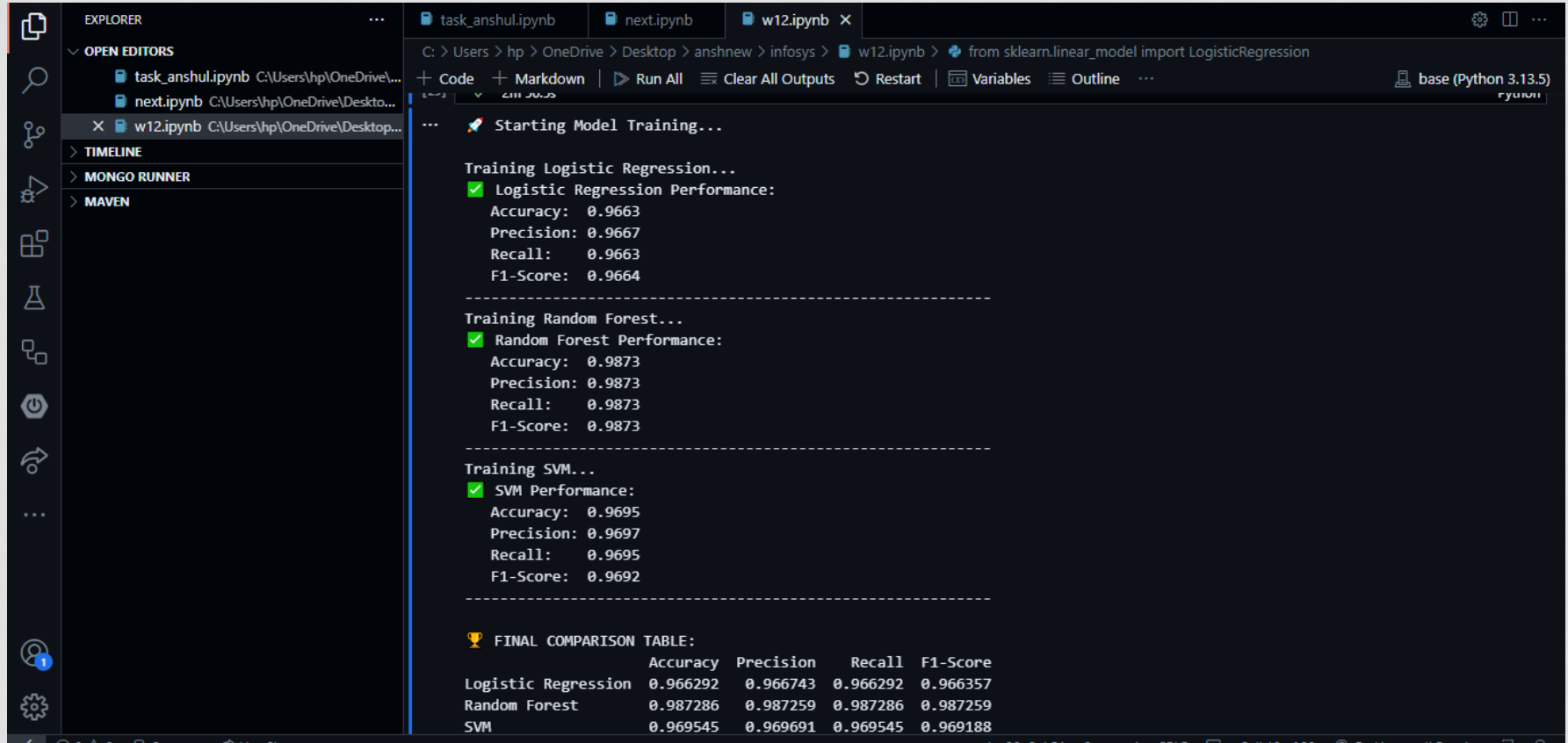
DATASETS USED

- NSL-KDD Dataset
- CICIDS2017 Dataset
- Publicly available network intrusion datasets



SYSTEM ARCHITECTURE

- Data Collection → Preprocessing → Feature Engineering → ML Models → Alerts



The screenshot displays a Jupyter Notebook environment with three open files: `task_anshul.ipynb`, `next.ipynb`, and `w12.ipynb`. The active file, `w12.ipynb`, is located at `C:\Users\hp>OneDrive>Desktop>anshnew>infosys>w12.ipynb` and uses the `base (Python 3.13.5)` kernel. The notebook content shows the import of `LogisticRegression` from `sklearn.linear_model` and the execution of model training and performance evaluation for three models: Logistic Regression, Random Forest, and SVM. The results are displayed as text output with green checkmarks indicating successful training and performance metrics.

```
C: > Users > hp > OneDrive > Desktop > anshnew > infosys > w12.ipynb > from sklearn.linear_model import LogisticRegression
```

... Starting Model Training...

Training Logistic Regression...

✓ Logistic Regression Performance:

Accuracy: 0.9663
Precision: 0.9667
Recall: 0.9663
F1-Score: 0.9664

Training Random Forest...

✓ Random Forest Performance:

Accuracy: 0.9873
Precision: 0.9873
Recall: 0.9873
F1-Score: 0.9873

Training SVM...

✓ SVM Performance:

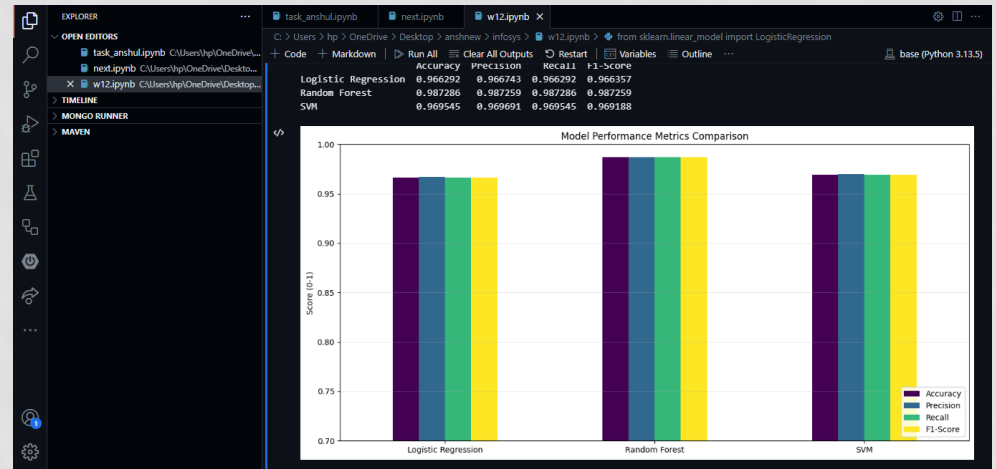
Accuracy: 0.9695
Precision: 0.9697
Recall: 0.9695
F1-Score: 0.9692

🏆 FINAL COMPARISON TABLE:

	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.966292	0.966743	0.966292	0.966357
Random Forest	0.987286	0.987259	0.987286	0.987259
SVM	0.969545	0.969691	0.969545	0.969188

MODULES

- Data Acquisition
- Preprocessing
- Feature Engineering
- Supervised Learning
- Unsupervised Learning
- Alert Generation



The screenshot shows a Jupyter Notebook interface with Python code for training and evaluating an Isolation Forest model. The code includes the following lines:

```
iso_preds_raw = iso_forest.predict(X_test)

# Convert to our format: 0 (Normal), 1 (Attack)
iso_preds = np.where(iso_preds_raw == 1, 0, 1)

print("Isolation Forest Results:")
print(classification_report(y_test_binary, iso_preds, target_names=['Normal', 'Anomaly']))
print("-" * 50)
```

The output of the code shows the training progress and the Isolation Forest Results:

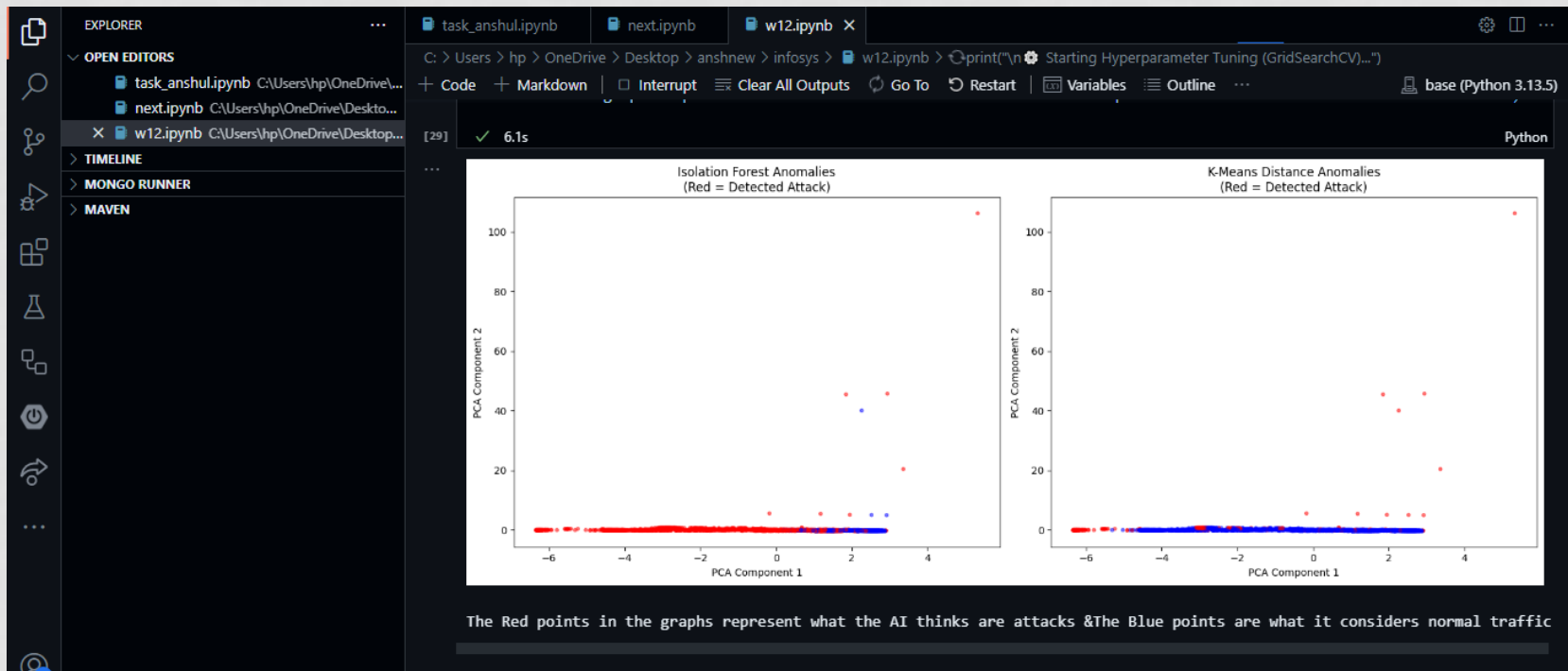
```
Training Isolation Forest...
Isolation Forest Results:
precision    recall  f1-score   support

Normal      0.80     0.91     0.85     1459
Anomaly     0.92     0.83     0.87     1923

accuracy          0.86
macro avg         0.86
weighted avg      0.87
```

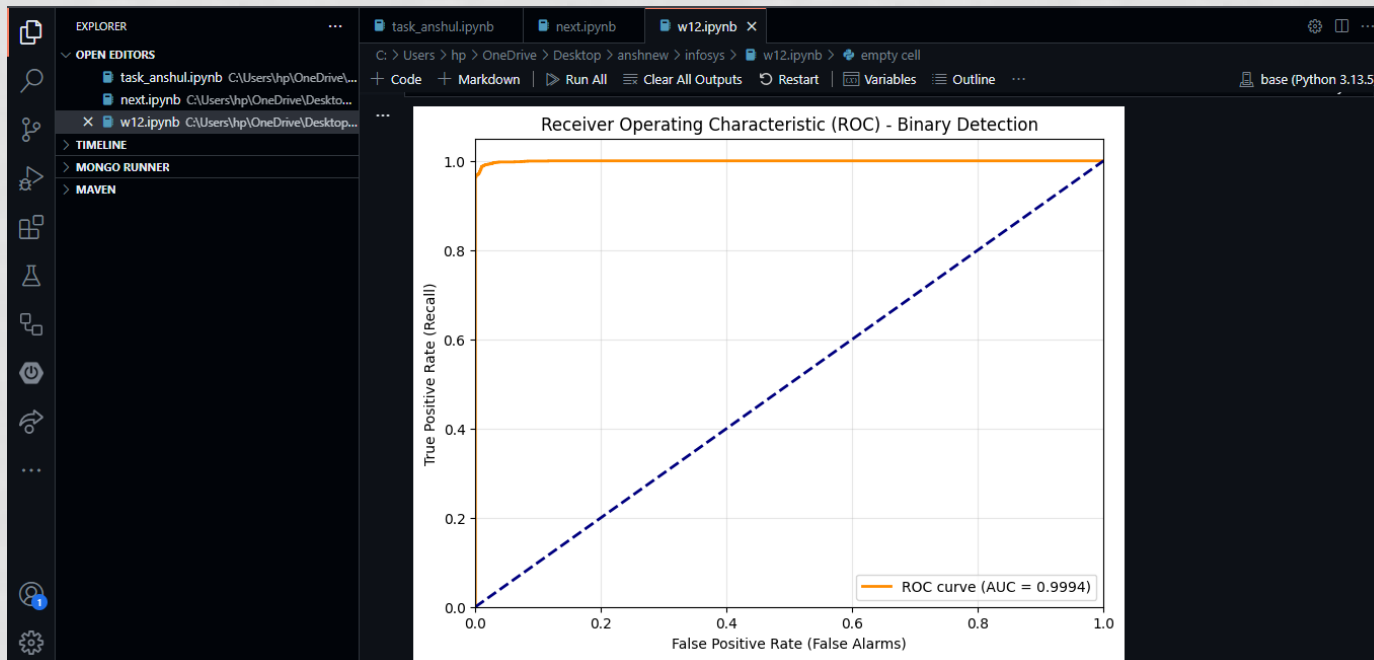
MACHINE LEARNING MODELS

- • Random Forest
- • Support Vector Machine
- • Logistic Regression



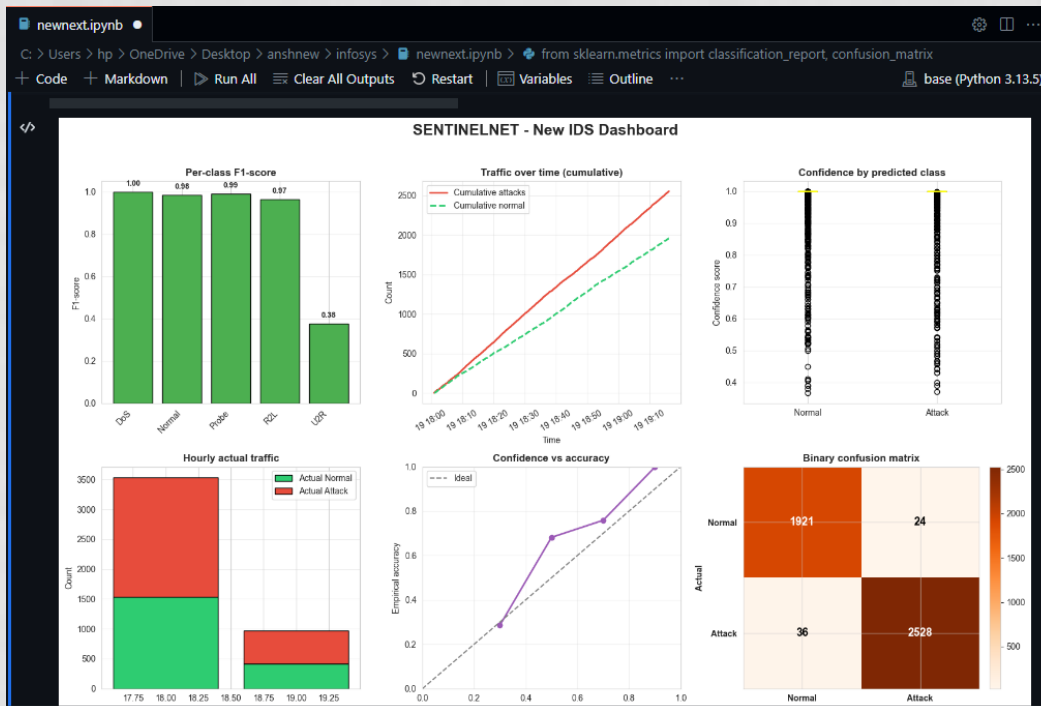
EVALUATION METRICS

- Accuracy
- Precision
- F1-Score
- ROC Curve



RESULTS & OUTCOMES

- Improved detection accuracy
- Reduced false positives
- Real-time alerts



CONCLUSION

- SentinelNet provides an intelligent, scalable, and effective solution for network intrusion detection.
- Github:<https://github.com/springboardmentor2507-alt/AI-Powered-NIDS/tree/Anshul-Bhilare>

